

思考题， Unix v6++ 系统复现除法计算器

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博

学号 2351520

一、divide.c - 不使用远跳转的实现

实现方式：使用标志位（flag）方式处理除零异常

核心机制：

- 使用 `signal(SIGFPE, sig_dyzero)` 注册信号处理函数
- 定义 `volatile int div_error` 标志位
- 信号处理函数仅设置标志位并打印错误信息
- 主循环在除法后检查标志位，决定是否打印结果

注：UnixV6++中有个 Bug，实际压栈的地址是除法指令后一条指令，所以信号处理函数返回后，程序会继续执行除法指令后的下一条代码。所以不能实现在 ubuntu 中出现的无限循环情况。不使用远跳转也可以实现正确的处理方式。

需要注意的是，在 UNIX V6++中，信号处理函数在被触发一次后，会自动恢复为默认处理方式（终止进程）。所以每次循环都需要在信号处理函数中重新注册信号。

Divide.c 程序：

```
#include <stdio.h>
#include <sys.h>

/* 标志位，指示是否发生除零错误 */
static volatile int div_error = 0;

/* 强制偶数地址对齐 */
__asm__(".align 2");
static void sig_dyzero(int signo)
{
    printf("Divide by zero!\n");
    signal(SIGFPE, sig_dyzero); /* 信号处理函数在被触发一次后，会自动恢复为
默认处理方式。所以需要重新注册信号，否则下次会直接退出 */
    div_error = 1;
}
int main1(void)
{
    int a, b;
    int c;
    char input[100];
    if (signal(SIGFPE, sig_dyzero) == -1)
        printf("can't catch divide by zero error!\n");
    for ( ; ; )
```

```

{
    printf("Input dividend:\n");
    gets(input);
    a = my_atoi(input);
    printf("Input divisor:\n");
    gets(input);
    b = my_atoi(input);
    div_error = 0;
    c = a / b;
    if (!div_error)
        printf("%d / %d = %d\n", a, b, c);
}
return 0;
}

```

虚拟机测试，可以看到输入除数为 0 后，程序提示一遍 Divide by Zero!，然后继续执行。

```

Bochs for Windows - Display
[/]#divide
Input dividend:
10
Input divisor:
2
10 / 2 = 5
Input dividend:
10
Input divisor:
0
Divide by zero!
Input dividend:
-

Process 1 finding dead son. They are Process 2 (Status:3) wait until
ess Exit! regs->eax = 4294967294 , u.u_error = 2
Process 0x2, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2>
> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x4
,0x410> <1029,0x411> <1030,0x412> <2047,0x413>

```

二、dividePro.c - 使用远跳转的实现

实现方式：使用 setjmp/longjmp 远跳转恢复执行流

核心机制：

- 使用 setjmp(fpe_env) 保存执行环境
- 信号处理函数中调用 longjmp(fpe_env, 1) 跳转回安全点
- 信号处理函数需重新注册

首先要在 lib 系统库中做 setjmp/longjmp 的实现：

```

jmp_buf 结构 (24 字节):
env[0] (偏移 0): ebx
env[1] (偏移 4): esi

```

```
env[2] (偏移 8): edi
env[3] (偏移 12): ebp (栈帧指针)
env[4] (偏移 16): esp (栈指针)
env[5] (偏移 20): eip (返回地址)
```

setjmp 函数 - 保存执行环境

```
/*
 * int setjmp(jmp_buf env)
 *
 * 功能: 保存当前执行环境到 env 中
 * 返回值: 首次调用返回 0; 由 longjmp 跳转回来时返回 longjmp 的 val 参数
 */
__asm__(
".globl _setjmp\n"
"_setjmp:\n"
"    movl 4(%esp), %eax\n"      /* eax = env, 获取 jmp_buf 指针参数 */
"    movl %ebx, 0(%eax)\n"     /* env[0] = ebx, 保存被调用者保存寄存器 */
"    movl %esi, 4(%eax)\n"     /* env[1] = esi */
"    movl %edi, 8(%eax)\n"     /* env[2] = edi */
"    movl %ebp, 12(%eax)\n"    /* env[3] = ebp, 保存栈帧指针 */
"    leal 4(%esp), %ecx\n"     /* ecx = esp+4, 即 setjmp 返回后调用者的 esp 值 */
"    movl %ecx, 16(%eax)\n"    /* env[4] = 调用者的 esp */
"    movl (%esp), %ecx\n"      /* ecx = 返回地址 (setjmp 调用后的下一条指令) */
"    movl %ecx, 20(%eax)\n"    /* env[5] = 返回地址 */
"    xorl %eax, %eax\n"        /* eax = 0, 首次调用 setjmp 返回 0 */
"    ret\n"                   /* 返回到调用者 */
);
```

longjmp 函数 - 恢复执行环境并跳转

```
/*
 * void longjmp(jmp_buf env, int val)
 *
 * 功能: 恢复 env 中保存的执行环境, 跳转回 setjmp 调用处
 * 参数: env - 由 setjmp 保存的环境; val - setjmp 的返回值(0 会被改为 1)
 */
__asm__(
".globl _longjmp\n"
"_longjmp:\n"
"    movl 4(%esp), %edx\n"      /* edx = env, 获取 jmp_buf 指针 */
"    movl 8(%esp), %eax\n"     /* eax = val, 获取返回值参数 */
"    testl %eax, %eax\n"       /* 测试 val 是否为 0 */
"    jnz 1f\n"                 /* 不为 0 则跳过下一条 */
1f:
```

```

"    incl %eax\n"                /* val 为 0 时改为 1, 因为 setjmp 首次返回 0 需区分 */
"1:\n"
"    movl 0(%edx), %ebx\n"        /* 恢复 ebx */
"    movl 4(%edx), %esi\n"        /* 恢复 esi */
"    movl 8(%edx), %edi\n"        /* 恢复 edi */
"    movl 12(%edx), %ebp\n"       /* 恢复 ebp 栈帧指针 */
"    movl 16(%edx), %esp\n"       /* 恢复 esp 栈指针 - 关键! 切换回 setjmp 调用者的栈 */
"    jmp *20(%edx)\n"            /* 跳转到 setjmp 的返回地址, eax 中的值作为 setjmp 返回值 */
);

```

dividePro.c 程序: 使用 setjmp/longjmp 保存和恢复执行环境, 从信号处理函数直接跳转到安全点, 成功处理除法异常, 循环执行程序

```

#include <stdio.h>
#include <sys.h>

static jmp_buf fpe_env;

/* Force even address alignment with asm nop */
__asm__(".align 2");

static void sig_dyzero(int signo)
{
    printf("Divide by zero!\n");
    /* Re-register signal handler (UnixV6++ resets it after each signal) */
    signal(SIGFPE, sig_dyzero);
    longjmp(fpe_env, 1);
}

int main1(void)
{
    int a, b;
    int c;
    char input[100];

    if (signal(SIGFPE, sig_dyzero) == -1)
        printf("can't catch divide by zero error!\n");

    for ( ; ; )
    {
        if (setjmp(fpe_env) == 0)
        {

```

```

        /* Normal execution path */
        printf("Input dividend:\n");
        gets(input);
        a = my_atoi(input);

        printf("Input divisor:\n");
        gets(input);
        b = my_atoi(input);

        c = a / b;
        printf("%d / %d = %d\n", a, b, c);
    }
    else
    {
        /* Jumped here from signal handler */
        printf("Recovered from divide by zero, continuing...\n");
    }
}

return 0;
}

```

虚拟机测试，可以看到在输入除数为 0 时，除了信号处理函数中打印一句 Divide by zero! 之外，还打印了一句 Recovered from divide by zero, continuing..., 证明成功远跳转。然后程序继续执行：

```

Bochs for Windows - Display
A: B: CD USER Copy Paste Snapshot CONFIG Reset Suspend Power
[/]#dividePro
Input dividend:
10
Input divisor:
2
10 / 2 = 5
Input dividend:
10
Input divisor:
0
Divide by zero!
Recovered from divide by zero, continuing...
Input dividend:
-

Process 1 finding dead son. They are Process 2 (Status:3) wait until cl
ess Exit! regs->eax = 4294967294 , u.u_error = 2
Process 0x2, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0x3
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <
> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E
,0x410> <1029,0x411> <1030,0x412> <2047,0x413>

```